

# ComplexGitSync 4.2.1– Getting Started

Nicolas Flipó

October 2, 2026

## Contents

|          |                                       |          |
|----------|---------------------------------------|----------|
| <b>1</b> | <b>Getting Started</b>                | <b>1</b> |
| <b>2</b> | <b>Prerequisites</b>                  | <b>1</b> |
| <b>3</b> | <b>Installation</b>                   | <b>2</b> |
| <b>4</b> | <b>Create or Check a Project Spec</b> | <b>2</b> |
| <b>5</b> | <b>Initialise the Workspace</b>       | <b>2</b> |
| <b>6</b> | <b>Inspect the Runtime State</b>      | <b>3</b> |
| <b>7</b> | <b>Run the Git Cycle</b>              | <b>3</b> |
| <b>8</b> | <b>Log Files</b>                      | <b>4</b> |
| <b>9</b> | <b>Next Steps</b>                     | <b>4</b> |

## 1 Getting Started

This chapter guides a new user through the Multi-repo Sync CLI. The documented entry point is `cgitsync`.

The primary lifecycle is:

1. `initialise(.cgs)` → clone all repositories → **READY**
2. `initialise(.gts)` → load or restore a saved snapshot → **READY**
3. `pull(.cgs|.gts)` → resynchronise an existing tree
4. `checkout / add / commit / push / tag` → tree-wide Git operations
5. `freeze-release` → commit, tag, write the next `.gts` snapshot and update the project-local `.lgr`

## 2 Prerequisites

- Python 3.11 or later.
- Git 2.30 or later available on `PATH`.
- Working Git authentication for every remote used by the workspace.

### 3 Installation

Install ComplexGitSync from source with the Pixi-managed environment:

```
$ git clone https://github.com/flipoyo/ComplexGitSync.git
$ cd ComplexGitSync
$ pixi install
$ pixi run cgitssync --help
```

### 4 Create or Check a Project Spec

A `.cgs` file is the local authoring spec for the repository tree. It normally contains only a project name and repository identifiers:

```
project = "CGSil1"
repos = [
  "gitlab:CGS_test/CGSil1",
  "gitlab:CGS_test/CGSil2",
  "github:flipoyo/CGSih1",
]
```

Each identifier uses `provider:owner/repository`. ComplexGitSync runs `PARSE -> NORMALIZE -> VALIDATE` and supplies deterministic defaults: `main` branches, `ssh`, `nested_config = auto`, and the repository name as its path. A unique repository matching the project name is mounted at `..`. Advanced tables are reserved for overrides.

Canonical providers are `github`, `gitlab`, `codeberg`, and `custom`. Their known hosts are `github.com`, `gitlab.com`, and `codeberg.org`; SSH and HTTPS remotes are generated automatically. Custom hosting requires an explicit `gitprovider_url` and never guesses a host. For example, `codeberg:GX4G/GX4G` identifies owner `GX4G` and repository `GX4G` on Codeberg.

Copy `examples/template.cgs` when starting a specification. It combines plain repository identifiers with one inline override. Developers can compare it with `examples/normalized_template.cgs`, which shows the complete canonical document after normalization and is not intended as the usual hand-authored form.

The common inline options are `default_branch`, `fallback_branch`, `access_protocol`, `relative_path`, and `nested_config`. A relative path is resolved from the parent repository. Nested config can be `auto` to load the sole root-level `*.cgs`, `disabled` to stop discovery, or a relative `.cgs` path to select an exact file inside the repository.

```
$ pixi run cgitssync validate install.cgs
$ pixi run cgitssync view-tree install.cgs
```

A valid declared tree prints a lifecycle line such as:

```
DECLARED ready=false complete=true
```

### 5 Initialise the Workspace

Use `initialise` as the canonical first command. From a `.cgs` file it clones the full repository tree, validates it, writes the runtime snapshot under `<workspace>/cgitssync/state/`, and leaves the tree `READY`.

```
$ pixi run cgitssync initialise install.cgs
```

The same canonical project definition can be authored directly. The `-repo` option is repeatable:

```
$ pixi run cgitssync initialise --project CGS111 \
  --repo gitlab:CGS_test/CGS111 \
  --repo codeberg:GX4G/GX4G
```

Use either a source file or `-project` plus `-repo`, never both. To write a `.cgs` without initializing a workspace, use `discover -write FILE` on an existing checkout, or `submodules init`.

When `-output-path` is omitted, `cgitssync` uses the workspace-level default `CGSPATH=../..`, then derives `CGSHOME=CGSPATH/<project-name>`. The explicit equivalent is:

```
$ pixi run cgitssync initialise install.cgs --output-path "$CGSPATH"
```

From a previously generated snapshot (resolved automatically via the project's `.lgr` register; pass the path explicitly only to override that discovery):

```
$ pixi run cgitssync initialise
```

Successful `.cgs` initialisation prints a readable operation sequence `GT-LOAD->GT-DISCOVER->GT-VALIDATE->` the explicit workflow `load->expand->validate->clone->gitignore`, the per-repository `git clone` action, the resolved root path, a report of any `.gitignore` changes, and a compact tree outline. Structured log records also put the operation code first, for example `GT-DISCOVER`, `GT-VALIDATE`, `GT-CLONE`, or `GT-GITIGNORE`. If a stale partial checkout blocks the clone step, or a destination holds work that exists nowhere else, `initialise` fails explicitly and changes nothing. There is no clean-up command: commit and push that work, or move those directories aside yourself, then run `initialise` again.

## 6 Inspect the Runtime State

```
$ pixi run cgitssync view-tree
$ pixi run cgitssync view-tree --depth 2 --collapse parent_2
$ pixi run cgitssync status
```

If a command accepts an optional snapshot and none is provided, `cgitssync` discovers the latest `.gts` automatically via the project's `.lgr` register under `CGSHOME/.cgitssync/`. Each generated `.gts` snapshot actually lives under its own content-addressed `CGSHOME/.cgitssync/state/` directory; pass an explicit path (e.g. `-gts FILE`, or a positional source, depending on the command) only to override that discovery. `CGSHOME` can be set explicitly; for `initialise(.cgs)` the default is `CGSPATH=../..`, and later commands can also discover the workspace by walking upward from the current directory.

## 7 Run the Git Cycle

Once the tree is `READY`, the Multi-repo Sync commands operate on every repository in deterministic order:

```
$ pixi run cgitssync pull
$ pixi run cgitssync checkout feature/my-branch
$ pixi run cgitssync add
$ pixi run cgitssync commit "feat: update project CGS#1"
$ pixi run cgitssync push
$ pixi run cgitssync freeze-release release-2026.05 "release 2026.05"
$ pixi run cgitssync checkout release-2026.05 --ref-kind tag
```

`pull` walks the tree parent-first, including the project root: `root -> parent -> leaf`. Mutation commands such as `add`, `commit`, `push`, and `freeze-release` walk the opposite direction:

leaf -> parent -> root. When local files block the safe pull, the CLI suggests `cgitsync pull -force`; that recovery command force-updates the same tree. It sets uncommitted and untracked work aside with `git stash push -u`, and refuses, before changing anything, while a commit exists that no remote holds.

Pass `-gts <snapshot.gts>` when you want to pin a command to an explicit snapshot, and pass `-dry-run` to add, commit, push, or freeze-release to preview the plan without mutating repositories.

## 8 Log Files

Every CLI command writes a timestamped log file. On Linux the default location is `/.local/state/ComplexG`; if `XDG_STATE_HOME` is set, logs are written under that state directory instead. The CLI prints the resolved path as `log_file=<path>`.

## 9 Next Steps

See Chapter ?? for the command reference, document formats, preflight checks, and troubleshooting notes.